

Ae

Definition

Ae is a set of text files that an agent works through to carry out an assignment. Each set with a purpose is an Ae. This page contains the rules.

1. Text only

An Ae contains only text, and nothing outside that text is part of it. No particular format or language is required: markdown, plain text, or whatever suits the purpose.

Nothing in the Ae executes. The text is read, and the agent that has read it acts. A link opens nothing by itself, and a script stored as content does not run until someone executes it from outside.

Non-text resources are represented through text: an image, a repository, a sensor, or a tool enters as text that points to it. The arrangement of that text adapts to its reader; it need not be presented for human reading and preserves the distinctions needed for interpretation.

2. Content and indexes

Everything in an Ae is content: information, rules, procedures, representations of external resources, and results of previous runs. There is no hierarchy among them.

Some of that content exists to lead to other content. We call this an index. An index can lead to other indexes.

Indexes are the only links between pieces of content. A file's location means nothing by itself, and content that no index leads to still exists, even if it is not found.

Index is a function, not a file type. The same text is an index for an agent passing through it and ordinary content for an agent coming to correct it.

3. DNA

DNA is the text with which a run begins: its objective, entry point, and closing conditions, given directly or discoverable from that entry point.

The entry point is the only content the agent reaches without an index leading to it; everything else is reached from there. It depends on the run, not the Ae, so two different DNAs can enter at different points and traverse different parts of the same content.

4. Life and death

Life is an agent's run directed toward an objective. Death is the closing of that run under the conditions set by its DNA, whether or not the objective is achieved.

Death is total: the next run inherits no context, memory, or state. Only what was written as content persists from a life. An assignment after closing requires a new run.

Rules and design

These four are the rules. How to classify content, what scope to give each agent, which model to use, and how to distribute files are design decisions, with more than one possible answer.

Consequences

The rules do not say how to classify content, what must be written at closing, what each agent can reach, or what starts the next run. These decisions still have to be made. Those on this page are the ones that arise immediately.

Classification

Content is classified with tags so that indexes can work. An index that gives only names forces the agent to open each piece of content to find out what is inside. One that provides the classification lets it choose before opening.

The better the content is classified, the denser the indexes can be, and the more can be automated around them.

The same content can appear in several indexes—a task by project, urgency, and date—and an index of indexes lets the agent choose a route before opening the material.

Each design defines its tag vocabulary and the conditions for assigning tags. Someone must keep

them up to date: the agent that writes, at closing, or an agent that periodically organizes the content.

Where the files live makes no difference, because the structure is in the indexes. A single folder is enough.

What is written at closing

Only what is written persists from a life, so whatever must continue has to be written: the result, the pending state, and condensed conclusions under tags that will make them discoverable. DNA decides when and in what form.

The four states of content

For each agent, content can be in one of four states. The design chooses which.

State	What the agent sees	What it can do
Direct	The entire content	Use it as is: it already serves its purpose as text
With a path	The content and the path to the resource it represents	Reach the resource under that path's rules or approvals
Existence only	That the content exists	Nothing: it has no path
Hidden	Nothing	Nothing

A path here is the route to a resource outside the Ae—an account, a tool, a repository, a sensor—not an index.

Representing a resource does not confer the ability to use it. An Ae can describe a robot precisely without being able to move it: the text indicates how to locate it, and the environment provides execution. The text states the condition of use; the environment enforces it.

What starts the next run

The rules describe the closing of one life, not the beginning of the next. A human, a schedule, an event, or another agent's closing can start it. This enables continuous operation without keeping any conversation alive.

How to tell whether the design works

A design works better the less the agent needs beyond what is written. If an assignment succeeds only when the agent arrives with tools and knowledge already loaded, content is usually missing: something unwritten, a nonexistent index, or an incomplete representation.

It is worth testing with a bare model: no automatically invoked skills, no servers loaded by default, nothing but the DNA and what the Ae provides. This exposes those gaps. It is not a condition of use: an Ae is text, and any model can work through it.

Cases

The same route—DNA, indexes, content, execution—applies to very different materials. The content and tags change.

Domain	Content	Possible indexes
Personal life	Tasks, commitments, habits, reading	By date, status, or urgency
Relationships	Messages, conversations, contacts	By correspondent or subject
Projects	Objectives, tasks, owners, blockers	By project and status; next relevant work
Code	Changes, issues, reviews, active work	Assignments, work reservations, and results between agents
Knowledge	Books, articles, sources, research	By topic, evidence, or question
AI memory	Condensed conclusions, decisions, pending items	By topic or scope, retrieving only what is relevant
Business	Data, metrics, clients, calculation processes	By process, to operate spreadsheets and dashboards
Software	Skills, plugins, MCP, APIs, and conditions of use	By capability, obtaining only what is needed
Physical world	3D models, sensors, equipment, robotics	By resource and procedure, acting through available interfaces

These are possible applications, not experimental results.

One life

Someone saves what they read. Each text becomes content with tags reflecting what made it useful, and an index groups the texts by topic. The DNA assigns gathering what exists on a topic and writing it down. The agent enters, opens three or four pieces of content, writes the result, and closes. Five files, no permissions, no tools, and no schedules.

Chained lives

The Ae grows with a project: objectives, tasks, and blockers as content, and an index that sorts them by status. Each life takes a task, does it, writes the result

and what remains pending, and closes. The next enters without context and takes the next task.

The next agent needs the previous agent's conclusion, not its conversation. The conversation would grow without limit; the conclusion is content and can be found through its tags.

With capability

The Ae incorporates a representation of a spreadsheet and the calculation criteria. The DNA assigns updating the report. The agent calculates, updates the sheet through the environment, and records the operation.

If it encounters a specified condition, it creates a task in the index of the preceding project. It drafts the message that should be sent, but does not send it: the environment did not give it that capability.

Scale

What grows and what does not

	What appears	What starts it	What stays the same
One person	One Ae, a few assignments	The person	Content, indexes, DNA, life and death
Continuous attention	Runs that do not require someone to be present	A schedule, an event, or another run's closing	Content, indexes, DNA, life and death
Several Ae	Coordination that creates and governs the others	The coordinating Ae	Content, indexes, DNA, life and death
Self-improvement	New procedures and indexes made from existing pieces	An agent, with any approval the design requires	Content, indexes, DNA, life and death
Physical world	Sensors, equipment, and robotics as representations	The same as in any other case	Content, indexes, DNA, life and death

The last column is unchanged in every row. There is no point at which the architecture needs to change.

Why it can grow

No run inherits context, so a new agent need not know anything about those that have already worked: its DNA and an index are enough. Coordination cost does not grow with the number of agents.

The agent reads a route, not the whole set, so an Ae with a million pieces of content is no harder to traverse than one with a hundred, as long as the indexes are up to date.

Different scopes

An Ae can create others and give each what belongs to it.

	Knows the resource exists	Has the path	Needs approval
Operations Ae	Yes	Yes	No
Sales Ae	Yes	Yes	Yes
Development Ae	No	No	—

Conditions flow down from coordination, and authorized results flow back up to it. Copying an Ae does not define how it is synchronized: what is shared, what is updated, and what remains isolated must be decided.

Self-improvement

Procedures, indexes, and results are content, so the system can work on itself.

An agent can compose a new procedure from pieces already written, repair broken indexes, reclassify content, or propose changes to the design itself. An Ae can have the purpose of building others.

Maintaining the system is work, and work the system can take on. With enough design and testing, the

system maintains itself better than the person who built it would, because it does so continuously and across all the material at once.

What is automated, then, is improvement, and improvement includes whatever automations are needed to achieve it.

How far it reaches

Text can point to a sensor, a robotic arm, or a fleet just as it points to an API. The limit is what the environment can reach.

A person with one Ae and an organization with two hundred follow the same route.

Getting started

This final page addresses both the reader and their agent.

If you are the person: you do not need to understand the whole system to have one. Give these pages to your agent and let it begin. You decide what matters; it writes it down.

If you are the agent, what follows is your assignment.

You have read the four preceding pages. The person who gave them to you has not yet built anything with them.

Your assignment is not to explain this document or design a complete system. It is to find out with them what is worth building first, and build it today.

Start with their work. Ask where they repeat themselves, explain the same thing again, what they lose or reopen each week. One or two questions per turn.

Once you understand something concrete, propose the smallest useful Ae: what content it would hold, with which tags, what index would group it, and what a first DNA would say. Use their material.

Explain as you go. When using a term from the document, point to it in their own material instead of defining it.

Divide the work: they decide the criteria—what matters and what must be findable—and you apply those criteria by writing the tags, indexes, and DNA.

Tell them what you see. If it depends on the environment (permissions, tools, execution), say so. If a design will fail, say so before writing it.

Before accepting anything as sound, check whether a model with nothing preloaded could carry out that assignment using only what is written. If not, content is probably missing.

Finish with something that can run today, even if it is just five files. If they wish, carry out the first DNA yourself and write down what the next run will need.